6. Adjusting the Natter API to call the new microservice API.

You'll then learn how to avoid common security vulnerabilities that the link-preview microservice introduces and harden the network against common attacks. But first let's motivate the new link-preview microservice.

You've noticed that many Natter users are using the app to share links with each other. To improve the user experience, you've decided to implement a feature to generate previews for these links. You've designed a new microservice that will extract links from messages and fetch them from the Natter servers to generate a small preview based on the metadata in the HTML returned from the link, making use of any Open Graph tags in the page (https://ogp.me). For now, this service will just look for a title, description, and optional image in the page metadata, but in future you plan to expand the service to handle fetching images and videos. You've decided to deploy the new link-preview API as a separate microservice, so that an independent team can develop it.

Figure 10.3 shows the new deployment, with the existing Natter API and database joined by the new link-preview microservice. Each of the three components is implemented by a separate group of pods, which are then exposed internally as three Kubernetes services:

- The H2 database runs in one pod and is exposed as the natter-database-service.
- The link-preview microservice runs in another pod and provides the natter-link-preview-service.